

Institute of Business Administration (IBA)

Course: CSE 317: Algorithms: Design and Analysis (Spring 2026)

Instructors: Dr. Shahid Hussain, Dr. Taslim Murad

Project Milestone 1: Project Idea

Topic: Approximate String Matching with Edit Distance $\leq k$

Date: April 17, 2026

Group Members:

Abdullah Irfan	-	29266
Abdullah Khalid	-	28084
Kashish Anil Kumar	-	29105
Muskan Pawan	-	28394

Contents

1	Problem Statement and Formal Definition	1
1.1	Mathematical Representation	1
1.2	Illustrative Example	1
1.3	Real-World Utilization	2
2	Tentative Algorithms for Comparison	3
2.1	Baseline: Recursive Branch-and-Bound (Brute Force)	3
2.2	Dynamic Programming: Sellers' Algorithm	3
2.3	Dynamic Programming Optimization: Ukkonen's Banded DP	4
2.4	Divide and Conquer: Pigeonhole Principle Filter	4
2.5	Filtering Paradigm: Baeza-Yates Counting Filter	4
2.6	Bit-Parallelism: Wu-Manber Algorithm	4
2.7	Greedy Heuristic: Seed-and-Extend	5
2.8	Summary of Algorithmic Complexities	5
3	Methodology for Empirical Comparison	6
3.1	Testing Parameters	6
3.2	Metrics of Evaluation	6
4	References	7

1 Problem Statement and Formal Definition

Exact string matching algorithms (such as Knuth-Morris-Pratt or Boyer-Moore) are highly efficient but strictly mandate perfect character alignment. In fields dealing with noisy real-world data, such rigid constraints are insufficient. The approximate string matching problem expands upon classic substring search by locating a pattern within a larger text while tolerating a specific mathematical threshold of computational errors.

1.1 Mathematical Representation

Let Σ be a finite alphabet. Let the text $T = t_1 t_2 \dots t_n \in \Sigma^*$ be a string of length n , and the pattern $P = p_1 p_2 \dots p_m \in \Sigma^*$ be a string of length m .

Given an error threshold $k \in \mathbb{N}$, the objective of the approximate string matching problem is to find all ending positions j (where $1 \leq j \leq n$) in the text T such that there exists a starting position i (where $1 \leq i \leq j$) satisfying the condition:

$$d_E(P, T[i \dots j]) \leq k \quad (1)$$

Here, $d_E(X, Y)$ denotes the Levenshtein (edit) distance between strings X and Y . The edit distance is defined as the minimum number of single-character operations required to transform X into Y . The allowed operations are:

- **Substitution:** Replacing a character $a \in \Sigma$ with $b \in \Sigma$ (where $a \neq b$).
- **Insertion:** Inserting a character $a \in \Sigma$ into the string.
- **Deletion:** Removing a character from the string.

1.2 Illustrative Example

To demonstrate the evaluation of $d_E(P, T[i \dots j])$, consider a genomic sequence defined over the alphabet $\Sigma = \{A, C, G, T\}$.

Let the text be $T = \text{ATCGGTA}$ ($n = 7$), the search pattern be $P = \text{ATCGTA}$ ($m = 6$), and the maximum error threshold be $k = 1$.

Case 1: A Valid Match (Evaluating $T[1 \dots 7]$). Consider the entire text window $T[1 \dots 7] = \text{ATCGGTA}$. We align it with pattern P :

Pattern P :	A	T	C	G	--	T	A
Text T :	A	T	C	G	G	T	A

This alignment requires exactly 1 deletion (removing the extra G from T) to achieve a perfect match. Since $d_E = 1$, and $1 \leq k$, the algorithm correctly reports a match ending at position $j = 7$.

Case 2: An Invalid Match (Evaluating $T[1 \dots 6]$). Consider the substring $T[1 \dots 6] = \text{ATCGGT}$. We align it with pattern P :

Pattern P :	A	T	C	G	T	A
Text T :	A	T	C	G	G	T

Positions 1 through 4 match exactly. The final two positions reveal two distinct mismatches: at position 5, the pattern character T must be substituted with G; and at position 6, the pattern character A must be substituted with T. The optimal alignment therefore requires exactly 2 substitutions, giving $d_E = 2$. Since $2 > k$, the algorithm rejects this window.

1.3 Real-World Utilization

- **Genome Sequencing:** Hardware sequencers output millions of short DNA reads. These reads must be mapped back to a massive reference genome. Because sequencing machines possess an inherent error rate of approximately 1%, the mapping logic must tolerate k errors to successfully assemble the genome.
- **Variant Classification:** Bioinformatics pipelines rely on this error-tolerant mapping step to identify novel viral mutations and classify biological variants.
- **Fuzzy Text Search:** Backend systems and search engines utilise this computational logic to correct user typos and provide rapid query suggestions.

2 Tentative Algorithms for Comparison

We have compiled a comprehensive list of distinct algorithmic paradigms. For our final empirical evaluation, the group will select the Brute Force baseline and compare it against any 3 to 4 of the advanced algorithms listed below. This will allow us to contrast theoretical mathematical complexity against empirical runtime on synthetic and biological datasets.

2.1 Baseline: Recursive Branch-and-Bound (Brute Force)

This exhaustive recursive search serves as the algorithmic baseline for evaluating true Levenshtein distance. Unlike a naive sliding window (which is mathematically limited to Hamming distance substitutions), this approach recursively evaluates every possible edit operation at each alignment step. When a mismatch is encountered, the algorithm branches into three distinct recursive paths—substitution, insertion, and deletion—incrementing an accumulated error count along each branch. This constitutes a *branch-and-bound* strategy: any branch whose accumulated error exceeds k is immediately pruned and not explored further.

- **Time Complexity:** $O(3^m \times n)$ in the worst case. At each of the m alignment steps, the recursion may spawn up to 3 subproblems, yielding a ternary recursion tree of depth m per starting position in T . This bound is tight when $k \geq m$, because the threshold is then so permissive that no branch is ever pruned and the full tree is explored. For small values of k relative to m , aggressive pruning can substantially reduce average-case runtime; however, it does not improve the asymptotic worst-case bound.
- **Space Complexity:** $O(m + k)$. No explicit matrices are constructed; the maximum depth of the recursive call stack is bounded by the pattern length m plus at most k additional frames from insertion operations that advance in the text without consuming a pattern character.
- **Tradeoffs:** The algorithm requires minimal explicit memory, but its exponential worst-case runtime makes it infeasible for any non-trivial input. It serves as a correctness oracle: since its output is derivable directly from the definition of edit distance, all optimised algorithms will be cross-validated against it during testing.

2.2 Dynamic Programming: Sellers' Algorithm

This bottom-up approach, introduced by Sellers (1980) as an adaptation of the Wagner-Fischer string-to-string correction framework, computes the exact optimal alignment by building a 2D matrix. The key modification is the initialisation $D[0, j] = 0$ for all j , which permits the pattern to begin matching at any position in the text. Cell $D[i, j]$ then stores the minimum edit distance between the prefix $P[1 \dots i]$ and any suffix of $T[1 \dots j]$.

- **Time Complexity:** $O(n \times m)$
- **Space Complexity:** $O(m)$ when optimised (by maintaining only the current and previous columns of the matrix rather than the full $n \times m$ grid).
- **Tradeoffs:** It guarantees absolute optimality but is computationally unfeasible for genome-scale text data.

2.3 Dynamic Programming Optimization: Ukkonen's Banded DP

Because the objective is bounded by k errors, this optimisation calculates only a narrow diagonal band (of width $2k + 1$) within the Sellers DP matrix, entirely discarding cells that mathematically exceed the error threshold.

- **Time Complexity:** $O(n \times k)$
- **Space Complexity:** $O(k)$ for maintaining the active band of the current computational state.
- **Tradeoffs:** It maintains strict optimality while drastically reducing both time and space complexity for small values of k .

2.4 Divide and Conquer: Pigeonhole Principle Filter

If pattern P matches text T with an edit distance of at most k , dividing P into $k + 1$ non-overlapping segments guarantees (by the pigeonhole principle) that at least one segment must match the text exactly. The algorithm locates exact matches of these segments using a precomputed index, then applies limited Dynamic Programming only to candidate windows to verify whether the overall error is $\leq k$.

- **Time Complexity:** $O(n \times k)$ on average.
- **Space Complexity:** $O(m)$ to store the pattern segments and exact-matching precomputation tables.
- **Tradeoffs:** It aggressively prunes the search space to offer faster runtimes than pure DP while remaining mathematically exact.

2.5 Filtering Paradigm: Baeza-Yates Counting Filter

This algorithm acts as a rapid pre-processing filter by counting character frequencies within sliding text windows. Because edit distance permits insertions and deletions, a matching substring of T may have any length in the range $[m-k, m+k]$; the filter therefore examines windows of varying size within this range. It relies on a strict necessary condition: if the edit distance between a pattern and a text window is at most k , the sum of the positive differences in character counts between them cannot exceed k . If the total frequency discrepancy across all characters exceeds this threshold, the window is skipped without performing any sequence alignment.

- **Time Complexity:** $O(n)$ average-case scanning.
- **Space Complexity:** $O(|\Sigma|)$ to maintain the character count arrays, where $|\Sigma|$ is the size of the alphabet.
- **Tradeoffs:** It provides extremely fast text elimination but requires secondary DP verification for windows that pass the frequency test.

2.6 Bit-Parallelism: Wu-Manber Algorithm

This algorithm abandons standard array arithmetic and utilises the CPU's raw binary architecture. It encodes the pattern as bitmasks and simulates a Non-deterministic Finite Automaton (NFA) using bitwise hardware operations, maintaining $k + 1$ parallel bit-arrays to track varying error levels simultaneously.

- **Time Complexity:** $O(n \times k \times \lceil m/w \rceil)$, where w is the machine word size. When $m \leq w$, as is typical for short biological reads, the ceiling term collapses to 1, yielding an effective complexity of $O(n \times k)$ and enabling the entire pattern state to be updated in a single CPU word operation.
- **Space Complexity:** $O(|\Sigma| + k)$ to store the bitmasks for each alphabet character and the current operational states.
- **Tradeoffs:** It provides staggering empirical speedups for short biological reads (where $m \leq w$), executing multiple matrix updates in a single CPU cycle.

2.7 Greedy Heuristic: Seed-and-Extend

This algorithm queries short sub-strings (seeds) against a pre-built text index (such as a Suffix Array) to find exact matches instantly. Once a seed hit is located, the algorithm extends the match outward. If the error limit k is breached during extension, that search path is permanently abandoned.

- **Time Complexity:** Query time per read is roughly $O(m + \text{hits} \times m)$. This assumes a one-time offline index construction cost of $O(n)$ using optimal Suffix Array (SA-IS) or FM-index construction algorithms.
- **Space Complexity:** $O(n)$ overhead to construct and store the Suffix Array or FM-index of the reference text.
- **Tradeoffs:** It maps millions of reads in minutes but sacrifices worst-case optimality guarantees. It may permanently miss a valid alignment if sequence errors disrupt every chosen seed, though in practice it remains near-exact for short reads.

2.8 Summary of Algorithmic Complexities

Table 1 provides a high-level theoretical comparison of the algorithms proposed for this project.

Algorithm	Paradigm	Time Complexity	Space	Optimality
Recursive Brute Force	Branch-and-Bound	$O(3^m \times n)$	$O(m + k)$	Exact
Sellers' Algorithm	Dynamic Prog.	$O(n \times m)$	$O(m)$	Exact
Ukkonen's Banded DP	Optimised DP	$O(n \times k)$	$O(k)$	Exact
Pigeonhole Principle	Divide & Conquer	$O(n \times k)$ (avg)	$O(m)$	Exact
Counting Filter	Filtering	$O(n)$ (avg)	$O(\Sigma)$	Filter
Wu-Manber	Bit-Parallelism	$O(n \times k \times \lceil m/w \rceil)$	$O(\Sigma + k)$	Exact
Seed-and-Extend	Greedy Heuristic	$O(m + \text{hits} \times m)^*$	$O(n)$	Approx.

* Denotes per-read query time. Assumes a one-time offline index construction cost of $O(n)$.

Table 1: Comparison of theoretical time/space complexities and optimality across paradigms.

3 Methodology for Empirical Comparison

To validate the theoretical complexities discussed in Section 2, we will conduct a rigorous empirical evaluation of the selected algorithms. All implementations will be written in **C++** and executed under identical hardware constraints to ensure an unbiased comparison.

3.1 Testing Parameters

Our evaluation will measure algorithmic performance across three primary axes:

1. **Text Length (n):** Scaling the reference text from small synthetic strings to larger biological datasets to observe growth rates.
2. **Pattern Length (m):** Modifying the query length to test the scalability of bit-parallel and DP approaches.
3. **Error Threshold (k):** Gradually increasing the edit distance allowance to measure performance degradation in bounded and filtering algorithms.

3.2 Metrics of Evaluation

For each test case, we will record the following metrics:

- **Execution Time (ms):** The total wall-clock time required to successfully identify all valid matches.
- **Empirical vs. Theoretical Growth Rate:** For each algorithm, we will plot execution time against increasing n , m , and k to verify whether the observed growth rate matches the theoretical complexity class predicted in Table 1.

4 References

1. R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” *Journal of the ACM (JACM)*, vol. 21, no. 1, pp. 168–173, 1974.
2. E. Ukkonen, “Algorithms for approximate string matching,” *Information and Control*, vol. 64, no. 1–3, pp. 100–118, 1985.
3. S. Wu and U. Manber, “Fast text searching allowing errors,” *Communications of the ACM*, vol. 35, no. 10, pp. 83–91, 1992.
4. R. Baeza-Yates and C. H. Perleberg, “Fast and practical approximate string matching,” *Information Processing Letters*, vol. 59, no. 1, pp. 21–27, 1996.
5. H. Li and R. Durbin, “Fast and accurate short read alignment with Burrows-Wheeler transform,” *Bioinformatics*, vol. 25, no. 14, pp. 1754–1760, 2009.
6. P. H. Sellers, “The theory and computation of evolutionary distances: Pattern recognition,” *Journal of Algorithms*, vol. 1, no. 4, pp. 359–373, 1980.